

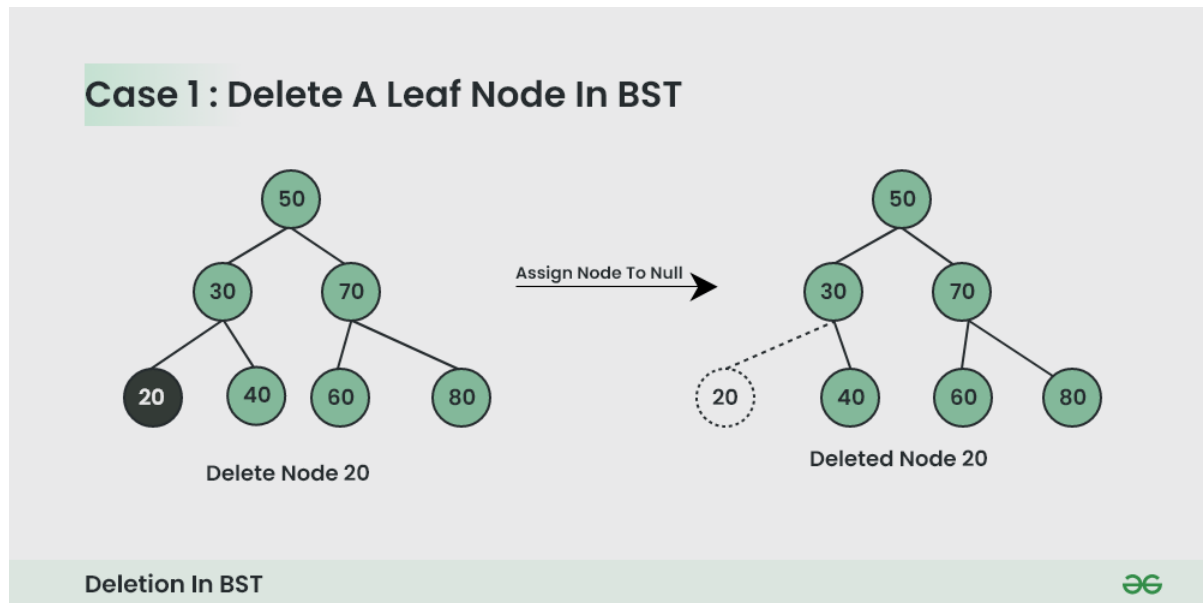
Deletion in Binary Search Tree (BST)

Last Updated : 05 Dec, 2024



Given a **BST**, the task is to delete a node in this **BST**, which can be broken down into 3 scenarios:

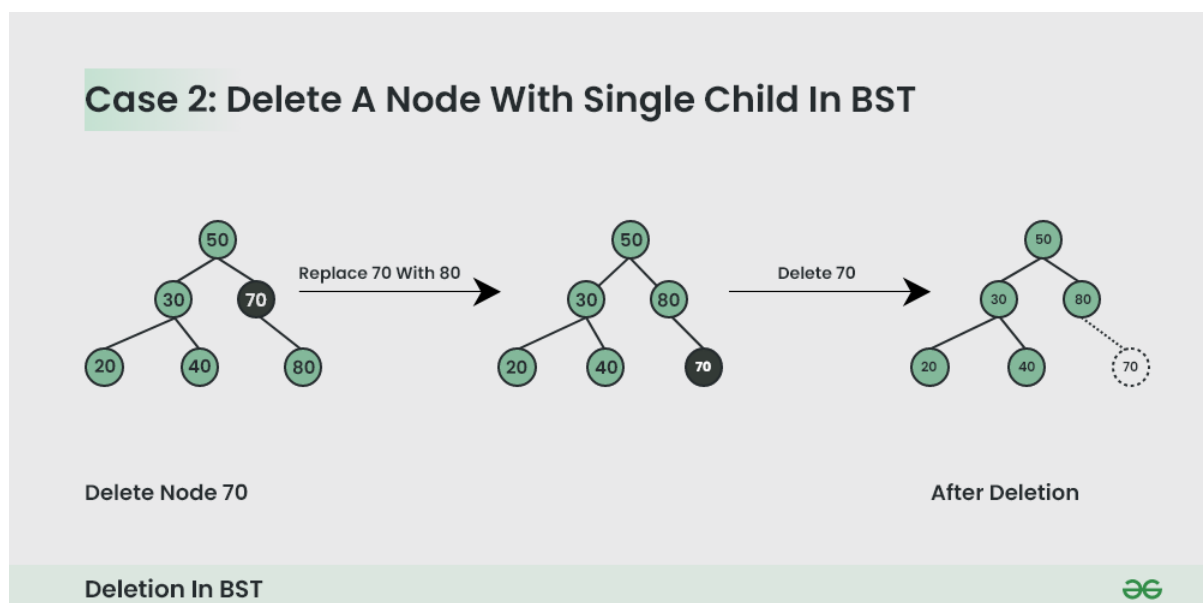
Case 1. Delete a Leaf Node in BST



Deletion in BST

Case 2. Delete a Node with Single Child in BST

*Deleting a single child node is also simple in BST. **Copy the child to the node and delete the node.***

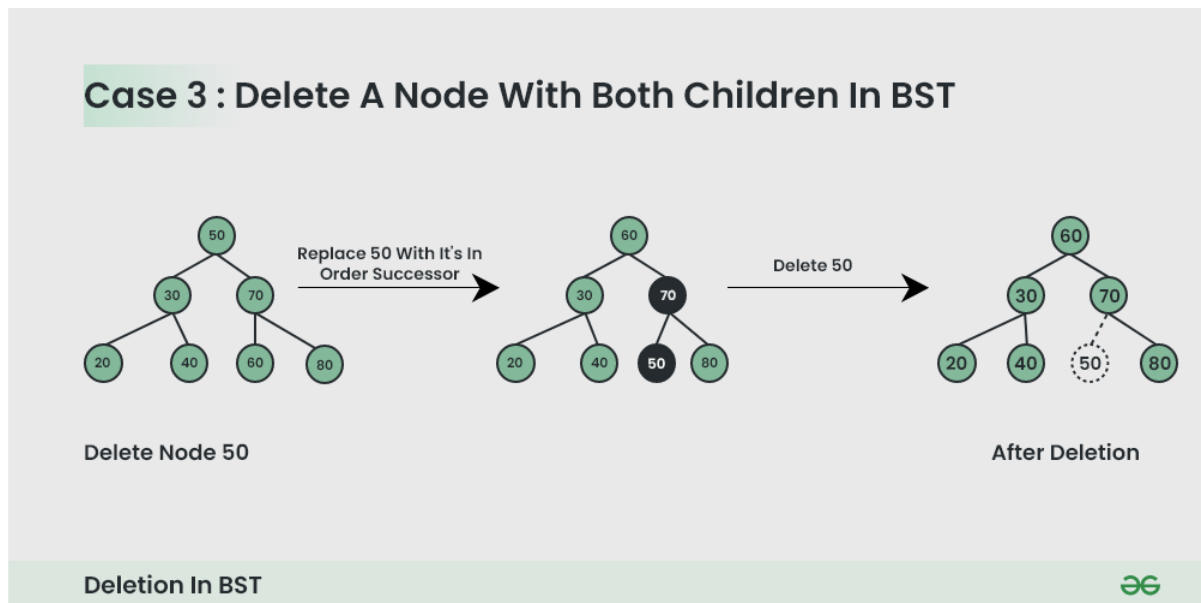


Case 3. Delete a Node with Both Children in BST

Deleting a node with both children is not so simple. Here we have to **delete the node** in such a way, that the resulting tree follows the properties of a **BST**.

The trick is to find the inorder successor of the node. Copy contents of the inorder successor to the node, and delete the inorder successor.

Note: Inorder predecessor can also be used.



Deletion in Binary Tree

Note: Inorder successor is needed only when the right child is not empty. In this particular case, the inorder successor can be obtained by finding the minimum value in the right child of the node.

Recursive Implementation of Deletion operation in a BST:

C++

C

Java

Python

C#

JavaScript

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
struct Node {
    int key;
    Node* left;
    Node* right;
    Node(int k){
```

```

        key = k;
        left = right = nullptr;
    }
};

// Note that it is not a generic inorder
// successor function. It mainly works
// when right child is not empty which is
// the case we need in BST delete
Node* getSuccessor(Node* curr){
    curr = curr->right;
    while (curr != nullptr && curr->left != nullptr)
        curr = curr->left;
    return curr;
}

// This function deletes a given key x from
// the given BST and returns modified root of
// the BST (if it is modified)
Node* delNode(Node* root, int x){

    // Base case
    if (root == nullptr)
        return root;

    // If key to be searched is in a subtree
    if (root->key > x)
        root->left = delNode(root->left, x);
    else if (root->key < x)
        root->right = delNode(root->right, x);

    // If root matches with the given key
    else {

        // Cases when root has 0 children
        // or only right child
        if (root->left == nullptr) {
            Node* temp = root->right;
            delete root;
            return temp;
        }
    }
}

```

```

        // When root has only left child
        if (root->right == nullptr) {
            Node* temp = root->left;
            delete root;
            return temp;
        }

        // When both children are present
        Node* succ = getSuccessor(root);
        root->key = succ->key;
        root->right = delNode(root->right, succ->key);
    }
    return root;
}

// Utility function to do inorder
// traversal
void inorder(Node* root){
    if (root != nullptr) {
        inorder(root->left);
        cout << root->key << " ";
        inorder(root->right);
    }
}

int main(){
    Node* root = new Node(10);
    root->left = new Node(5);
    root->right = new Node(15);
    root->right->left = new Node(12);
    root->right->right = new Node(18);
    int x = 15;

    root = delNode(root, x);
    inorder(root);
    return 0;
}

```

Output

5 10 12 18

Time Complexity: $O(h)$, where h is the height of the BST.

Auxiliary Space: $O(h)$.

The above recursive solution does two traversals across height when both the children are not NULL. We can optimize the solution for this particular case. Please refer [Optimized Recursive Delete in BST](#) for details.

We can avoid extra $O(h)$ space and recursion call overhead with iterative solution. The iterative solution has same time complexity but works more efficiently.

[Iterative Implementation of Deletion operation in a BST:](#)

Delete a node from BST | DSA Problem

 Comment

More info 

Advertise with us

Next Article 

Binary Search Tree (BST) Traversals –
Inorder, Preorder, Post Order

Similar Reads

Binary Search Tree

A Binary Search Tree (or BST) is a data structure used in computer science for organizing and storing data in a sorted manner. Each node in a Binary Search Tree has at most two children, a left child and a right...

 3 min read

Introduction to Binary Search Tree

Binary Search Tree is a data structure used in computer science for organizing and storing data in a sorted manner. Binary search tree follows all properties of binary tree and for every nodes, its left subtree...

 3 min read

Applications of BST

Binary Search Tree (BST) is a data structure that is commonly used to implement efficient searching, insertion, and deletion operations along with maintaining sorted sequence of data. Please remember th...

 3 min read

Applications, Advantages and Disadvantages of Binary Search Tree

A Binary Search Tree (BST) is a data structure used to storing data in a sorted manner. Each node in a Binary Search Tree has at most two children, a left child and a right child, with the left child containing...

🕒 2 min read

Insertion in Binary Search Tree (BST)

Given a BST, the task is to insert a new node in this BST. Example: How to Insert a value in a Binary Search Tree: A new key is always inserted at the leaf by maintaining the property of the binary search...

🕒 15 min read

Searching in Binary Search Tree (BST)

Given a BST, the task is to search a node in this BST. For searching a value in BST, consider it as a sorted array. Now we can easily perform search operation in BST using Binary Search Algorithm. Input: Root of...

🕒 7 min read

Deletion in Binary Search Tree (BST)

Given a BST, the task is to delete a node in this BST, which can be broken down into 3 scenarios: Case 1. Delete a Leaf Node in BST Deletion in BST Case 2. Delete a Node with Single Child in BST Deleting a...

🕒 10 min read

Binary Search Tree (BST) Traversals – Inorder, Preorder, Post Order

Given a Binary Search Tree, The task is to print the elements in inorder, preorder, and postorder traversal of the Binary Search Tree. Input: A Binary Search Tree Output: Inorder Traversal: 10 20 30 100 150...

🕒 10 min read

Balance a Binary Search Tree

Given a BST (Binary Search Tree) that may be unbalanced, the task is to convert it into a balanced BST that has the minimum possible height. Examples: Input: Output: Explanation: The above unbalanced BST...

🕒 10 min read

Self-Balancing Binary Search Trees

Self-Balancing Binary Search Trees are height-balanced binary search trees that automatically keep the height as small as possible when insertion and deletion operations are performed on the tree. The heigh...

🕒 4 min read
